SENSOR FUSION ALGORITHMS AND TRACKING FOR AUTONOMOUS SYSTEMS: UNMANNED GROUND VEHICLE (UGV)

Fall 2022 – Spring 2023

Michael Gee - Elizabeth Juarez - Hanson Nguyen - Kristopher Ottinger - Ryan Ramos - Dylan Senarath - Zoe Sy - Elvis Tang - Tan Tran - Christina Zavala - Tristan Williams

Faculty Advisor: Zekeriya Aliyazicioglu

Sponsor: Lockheed Martin

Abstract

Sensors are fundamental to an autonomous machine's understanding of its surroundings. Environmental data provided by sensors provide feedback from which the machine can make navigational decisions. The performance of each sensor, and integration with other sensors and sensor types, can determine the safety and feasibility of the autonomous machine. Each type of sensor has its own strengths and weaknesses; Integrating data from different sensors through sensor fusion, as opposed to using a single sensor, increases the system's accuracy, comprehensiveness, and performance. Sensor fusion also includes algorithmic techniques such as a Kalman Filter which allows for data stabilization due to variations in sensor measurements.

This project used sensor fusion to generate reliable and accurate sensor feedback to drive an autonomous ground vehicle. Due to variable and challenging environmental terrain, it is necessary to use multi-sensor inputs and data processing algorithms to accurately interpret the environment and minimize statistical sensor error. To develop an autonomous vehicle for difficult terrain, LIDAR, camera, sonar, and various other sensors were used in conjunction with experimental algorithms to improve the object detection and maneuvering of the vehicle. During experimental trials, the following elements were considered: object detection and positional accuracy. The combination of sensors used are the following: LIDAR with an IMU for distance mapping, a stereo camera for object visualization, and ultrasonic sensors for avoiding low to the ground obstacles. The type of fusion used in this project was complementary fusion. This type of fusion, with different sensor perspectives, overcomes the limitations of each sensor type.

Team Members:

- Michael Gee: mmgee@cpp.edu
- Elizabeth Juarez: eajuarez1@cpp.edu
- Hanson Nguyen: hansonnguyen@cpp.com
- Kristopher Ottinger: kbottinger@cpp.edu
- Ryan Ramos: ryanramos@cpp.edu
- Dylan Senarath: dnsenarath@cpp.edu
- Zoe Sy: zgsy@cpp.edu
- Elvis Tang: etang@cpp.edu
- Tan Tran: tantran@cpp.edu
- Christina Zavala: christinaz@cpp.com
- Tristan Williams tristanw@cpp.edu

Faculty Advisor:

- Dr. Zekeriya Aliyazicioglu: zaliyazici@cpp.edu

Sponsor:

- Lockheed Martin Corporation

Table of Content

1. Introduction

To explain sensor fusion, first a sensor must be defined. A sensor is a device that detects and responds to physical stimuli (such as heat, light, sound, pressure, magnetism, or a particular motion) and transmits the signal to an instrument for measurement or control. Everyday examples include temperature sensors that help with temperature regulation in buildings and fitness trackers on watches that use sensors to measure steps and heart rates. Examples in industry include pressure sensors used to monitor pipelines and alert centralized computing systems of potential leaks and irregularities, proximity sensors used to detect intruders or track crowd flow in crowded areas, and level sensors used for real time measurements of containers, bins, and tanks that are utilized to manage anything from waste, irrigation, and fuel. Then there are more complex sensors such as the AHRS or *Attitude and Heading Reference Systems* that consists of sensors on three axes such as roll pitch and yaw. [1]

1.1. Survey of Literature

While sensor data can be useful on their own, there is a much wider range of possibilities when they are combined through sensor fusion. Sensor Fusion is the "*combining of sensory data or data derived from sensory data such that the resulting information is in some sense better than would be possible when these sources were used individually*" [2]. By combining each sensor's strengths and compensating for any weaknesses via sensor fusion, the resulting information is significantly more accurate than any information from an individual sensor.

There are many great benefits to achieving this technology. Autonomous vehicles could reduce the number of road accidents caused by human error or negligence, for it could automatically stop the vehicle before it impacted another car or object. Autonomous vehicles also have a very important role in the military because this technology could allow dangerous tasks to be carried out by an autonomous robot rather than a human being. For instance, a mine sweeping instead of using a human to try to find mines a robot equipped with sensors could find the mines, making less dangerous error margin.

Kalman filtering [3], or linear quadratic estimation (LQE) is an algorithm used in sensor fusion that uses a series of measurements observed over time, including statistical noise and other inaccuracies, to make an educated guess about what the system is likely to do next by estimating a joint probability distribution over the variable for each time frame. In the case of autonomous vehicles, Kalman filters can be used to predict the next set of actions that the car should take based on the prior state. This is suitable for the project due to the following factors.

- It can predict the next state based on the previous state only.
- Computationally very fast making it well-suited for real time problems.
- In case of noise/error/uncertainty in the environment, Kalman filter will give a good result using Gaussian distribution.

The project was to develop an autonomous vehicle that uses a combination of sensors to detect proximity and distance measurement [4] via Ultrasonic Sensors, Lidar, and Camera. Ultrasonics would detect low lying objects, the Lidar would detect high level objects, and the camera would detect, identify, and track objects. All the data is then processed via sensor fusion in the form of a Kalman Filter where the data undergoes a complementary fusion. This type of fusion, along with different sensor perspectives, overcomes the limitations of each sensors' range.

The goal is to develop and implement advanced techniques for integrating data from multiple sensors to develop a coherent understanding of the environment. The goal is to design algorithms for autonomous systems that enable reliable and precise tracking and prediction of objects and events in complex and dynamic environments. Ultimately, it is desirable to enhance the safety, efficiency, and effectiveness of autonomous systems across a range of applications.

Sensor fusion plays an important role in the creation of any autonomous vehicle; It is key to an autonomous vehicle being self-reliant as the process provides the car with the necessary data to move around in its environment. While the concept is clear, as of this project, achieving complete autonomy is still in development. Tesla vehicles are a famous example; Its autopilot system allows the vehicle to drive itself in most situations. Their vehicles gain data about their surroundings with the use of radar, cameras, and ultrasonics and its autopilot AI filters processes that data to decide how the vehicle should drive itself. Despite all the work and resources put into the vehicles, there are still limitations on the vehicle's self-driving abilities with a reported 516 crashes from July to November of 2021 alone. [5]

1.2. Outline of Projects

Chapter 2 contains the hardware and software used in the project along with a short description of how they work. Chapter 3 contains the results and analysis of the project. The conclusion can be found in chapter 4 along with suggested future works if the project was to ever continue. The Final chapter contains the reference of sources as well as the code used in the project.

1.3. Project and Team Activities

This report contains the undergraduate student's results. This report includes the hardware and software used to create the UGV (Unmanned Ground Vehicle), the vehicle's design and schematics, and the source code. Also included are further implementations such as methodology, algorithms, and considerations for further research.

Prior to testing, research was done by looking into the algorithm (i.e. Kalman Filter) and the research literature on the topic of sensor fusion, UGVs, and the types of sensors used to gain understanding of the project.

The first stage of testing was done on the main sensors. In this stage investigations were done with regards to the strengths and limitations of the lidar, camera, and ultrasonics, and how to consider that data into the decision-making algorithm. During the second stage the team was divided into two groups: hardware and software. The hardware's focus was the battery-operated configuration of the chassis with all three sensors taking accurate measurements. Software's focus was to slowly integrate each sensor type into the sensor fusion algorithm starting from the sensor fusion of multiple ultrasonics to the addition of camera and lidar. Following this is the final stage where the sensors are integrated onto the chassis to take measurements of its surroundings and use that information to drive itself. During this stage it was discovered that the processing power available did not meet the requirements. This compounded with other issues in the project led to a scaled down version of the intended UGV.

2. Software and Hardware

2.1. Hardware

Below is the pertinent hardware used to create the UGV. It includes a list of sensors, microcontrollers, and a single board computer pertinent to the sensor fusion in this project.

2.1.1. Sensors

The following sections describe sensors used to collect data, the basics of how it works, and the specifications of each sensor.

2.1.1.1. Ultrasonics

Ultrasonic sensors, or ultrasonic transducers, are devices that generate or sense ultrasound energy. It sends out a sound or pulse, usually above the range of human hearing (ultrasonic), towards a target then measuring the time it takes for the sound echo to return. The operation of the ultrasonic sensor is represented in Figure 1. This is commonly used in underwater depth sounding, ultrasound in medicine, and in parking sensors to aid the driver when reversing into parking spaces.

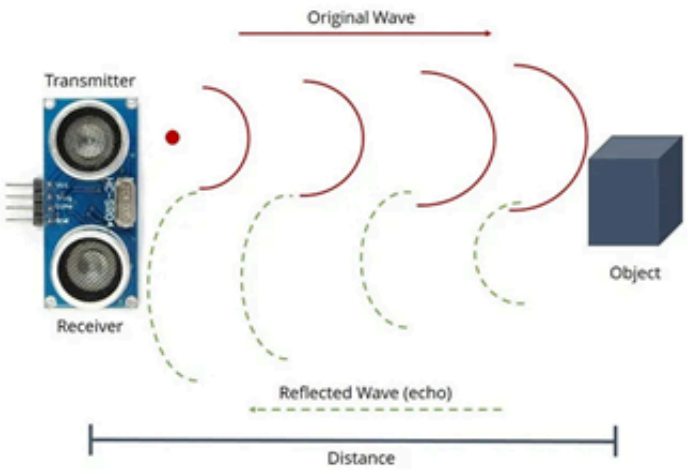

Figure 1: Diagram of Ultrasonic Sensor Operation [6]

There are several pros and cons to ultrasonic sensors. Ultrasonic Sensors rely on sound-based detection, so any object with a sound dampening effect or objects that are not entirely solid, such as soft fabrics or chain link fences, will render unreliable data. The model used in this project was the HC-SR04, as shown in Figure 2. This model was chosen because it was made for the Arduino system.

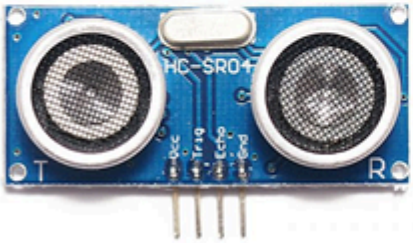

Figure 2: HC-SR04 Ultrasonic Sensor with a distance: 2cm~400cm; High precision: up to 0.3 cm; Detection angle: <15 ° [7]

2.1.1.2. Lidar

Lidar is an acronym for “Light detection and Ranging” or “laser imaging, detection, and ranging”. As the name suggests, it uses light to determine the range or distance of the object by measuring the time it takes for it to reflect to the receiver. This process is represented by Figure 4. As shown in Figure 3, the distance d is determined by the equation $d = (c * t) / 2$ where c is the *speed of light* and t is *time* spent for the laser to travel to the object then back at the detector. This distance data is thrown into a dot map where each dot is a reading in the lidar’s space, this is represented in Figure 6.

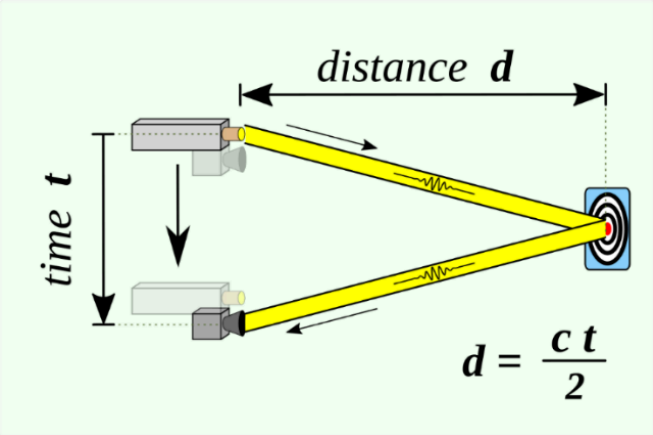

Figure 3: Diagram of Lidar Operation [8]

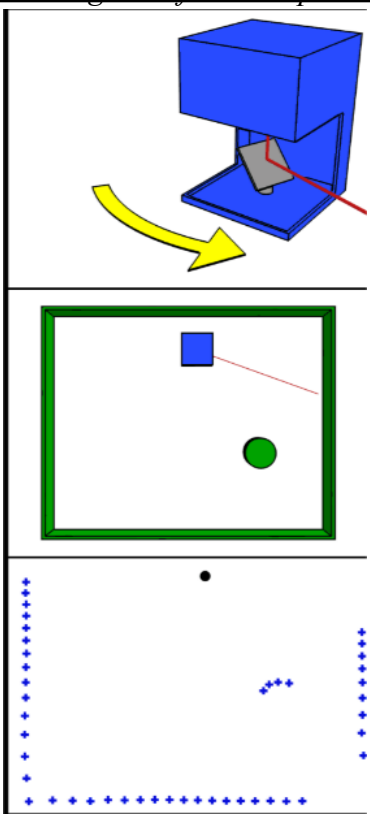

Figure 4: Diagram of how Lidar measures distance.

Lidar has several pros and cons. In terms of strengths, data can be collected quickly and with high accuracy. Surface data has a high sample density, it can be used day or night unlike the other sensors. Some cons to Lidar are its high cost, ineffectiveness due to bad weather such as rain or low clouds, reflective material, a very large dataset that can be difficult to interpret, and elevation issues. [9] The model used in this project is the Slamtec RPLIDAR A2M8 as shown in Figure 5.

Figure 5: Slamtec RPLIDAR A2M8 with a 360 degrees range, 2D Laser with 12 Meters Scanning Radius, and a scan rate from 5~15Hz. [10]

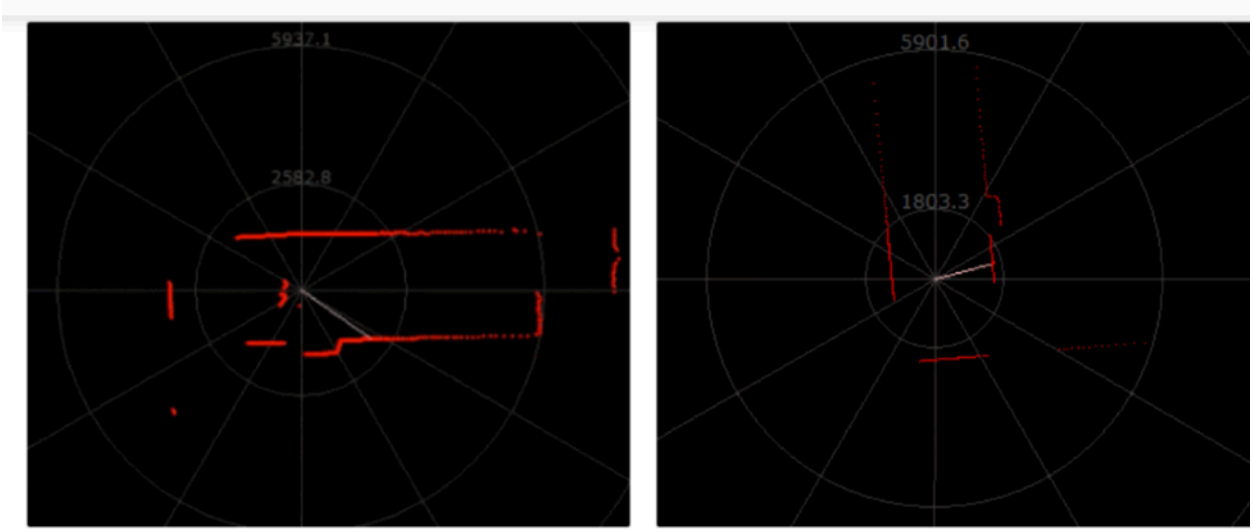

Figure 6: Example of Lidar Readings

2.1.1.3. Camera

The camera utilized within the project was the Arducam 5MP camera for Raspberry Pi which was the most suitable for the project at hand, as shown in Figure 7.

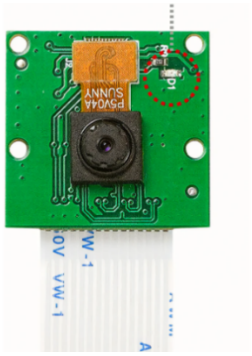

Figure 7: Arducam [11]

The small Raspberry Pi camera was useful for many reasons, but mostly due to its small size which barely took up any space on the car and the easy implementation within the Raspberry Pi. The camera has resolution specs of 2592 x 1944 for still pictures and max video resolution of 1080p. The camera utilizes a 5MPixel sensor with Omnivision OV5647 sensor in a fixed-focus lens. It has a 41 x 54-degree angle of view and a 2.0 x 1.3m field of view at 2m. It also has a maximum frame rate of about 30 frames per second.

2.1.1.4. IMU

Along with the use of Lidar, an IMU was used. IMU stands for Inertial measurement unit. It is an electrical device that measures and reports a body's specifical force, angular rate, and the body's orientation through the combined use of an accelerometer, gyroscope, and magnetometer. In short, an IMU works to detect changes in an object's pitch, roll, and yaw represented in Figure 8.

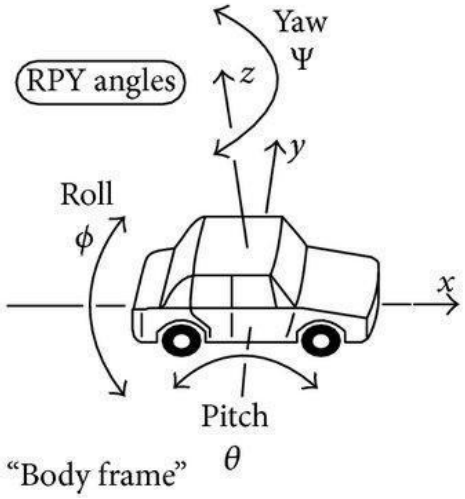

Figure 8: Diagram of IMU measurements [12]

A gyroscope is a device that measures angular rate, or the rate of rotation around an axis. It typically consists of a spinning rotor that is mounted in a set of gimbals, which allows the rotor to freely rotate around an axis perpendicular to the gimbals. As the rotor spins, it resists any change in its orientation, allowing it to maintain its angular momentum and providing a reference for measuring angular rate. In an IMU, gyroscopes are typically used to measure rotational motion and orientation, along with accelerometers which measure linear motion and gravitational force.

An accelerometer is a sensor that measures acceleration, or changes in velocity, in a specific direction. In the context of an IMU, it is used to measure linear acceleration, including both static and dynamic acceleration. Accelerometers are often used in conjunction with gyroscopes in an IMU since they provide complementary information about motion. While gyroscopes are good at measuring angular rate, accelerometers are good at measuring linear acceleration. By combining the data from both sensors, an IMU can provide a more complete understanding of a body's motion and orientation in three-dimensional space.

A magnetometer is a sensor that measures the strength and direction of a magnetic field. It is used to measure the Earth's magnetic field, which can be used as a reference for determining the orientation of the sensor in space. The Earth's magnetic field is relatively stable and consistent over large areas, making it a useful reference for navigation and orientation. By measuring the strength and direction of the magnetic field, a magnetometer can provide information about the orientation of the sensor with respect to the Earth's magnetic field.

2.1.1.5. Wheel encoders

Optical wheel encoders use a light source, a rotating encoder disc with slots, and photosensitive sensors. As the disc rotates, the slots pass between the light source and the sensors, changing the light intensity. Sensors detect these changes and convert them into electrical signals. By analyzing the signals from two sensor channels, position, speed, and direction of rotation can be determined. The wheel encoder used in this project was the Jeanoko Encoder meter wheels and its data was intended to dead reckon the AMR's pose in the SLAM algorithm. Due to wheel slippage wheel encoder data will drift as more distance is covered and so it is checked against the IMU's velocity data for a more accurate velocity calculation. [13]

2.1.2. Raspberry Pi

Raspberry pi is a series of small single board computers developed in the United Kingdom. Due to its low cost, modularity, and open design, it has a broad range of uses such as weather monitoring, and 2 other examples. It is typically used by computer and electronic hobbyists due to the adoption of HDMI and USB standards.

In this project the Raspberry Pi 4 Model B was used because it was one of the more advanced versions available that was capable of handling the computational power required, as shown in Figure 9. This model features a 1.5 GHz 64-bit quad core ARM Cortex-A72 processor, on-board 802.11ac Wi-Fi, Bluetooth 5, full gigabit Ethernet (throughput not limited), two USB 2.0 ports, two USB 3.0 ports, 1–8 GB of RAM, and dual-monitor support via a pair of micro-HDMI (HDMI Type D) ports for up to 4K resolution.

Figure 9: Raspberry Pi 4 B [14]

2.1.3. Arduino

Arduino is a popular open-source platform for building electronic projects. It consists of a microcontroller board and a development environment for writing code to control the board. The heart of an Arduino is a microcontroller, which is a tiny computer chip that can be programmed to perform specific tasks. The microcontroller on an Arduino is programmed using a simplified version of the C programming language. [15]

To use an Arduino, connect it to a computer using a USB cable and write code in the Arduino development environment. The code is then uploaded to the microcontroller on the board, which executes it and controls the hardware connected to it.

The hardware that can be controlled by an Arduino includes sensors, motors, lights, and other electronic components. These components are connected to the board using pins/wires and can be programmed to respond to different inputs and perform specific actions.

The Arduino Mega is a microcontroller board based on the ATmega2560, as shown in Figure 10. It has a total of 54 digital input/output pins, 16 analog inputs, and 4 hardware serial ports. Here is a breakdown of the pinouts:

Digital Pins:

- Pins 0-53 are digital input/output pins that can be used for digital input or output.
- Pin 51 – Right Encoder
- Pin 53 – Left Encoder
- Pins 0-21 are also analog input pins.
- Pin 11 – Left Motor
- Pin 3 – Right Motor
- Pin 4 – Trigger Pin Center
- Pin 5 – Trigger Pin Left
- Pin 6 – Trigger Pin Right
- Pin 7 – Echo Pin Center
- Pin 8 – Echo Pin Left
- Pin 9 – Echo Pin Right

Analog Pins:

- Pins A0-A15 (or 22-37) are analog input pins that can measure voltage between 0 and 5 volts.

Other Pins:

- Pins 14 and 15 are used for an external crystal oscillator and are not available for general use.
- Pins 16 and 17 are used for hardware serial communication with the computer and cannot be used for general input/output.

- Pins 18 and 19 are also used for hardware serial communication but can be used for general input/output if necessary.
- Pins 20 and 21 are reserved for the I2C interface and can also be used for general input/output.
- Pins 22-53 can be used for general input/output.

In addition to these pins, the Arduino Mega also has several other pins for power, ground, and other functions.

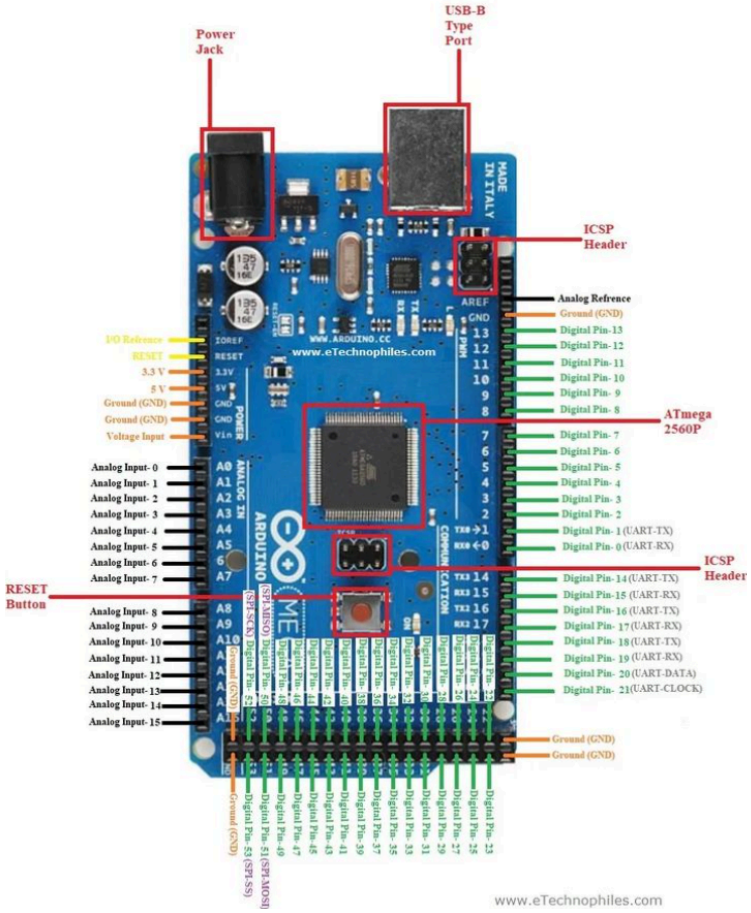

Figure 10: Arduino Mega

2.1.4. HB-25 Motor Controller

A single module was used to control each motor, requiring a single pulse width between 1000-2000 u-secs to set the motor speed. The model used in this project is the HB-25 as shown in Figure 11. The HB-25 had connections so that they could be used in parallel or series connection and has built in surge protection in the form of a 25-amp fuse. Each HB-25 is capable of 25 amps continuous current.

Figure 11: HB-25 Motor Controller

2.1.5. DC Brushed Motors

A pair of brushed motors were used to drive the chassis and combined are capable of moving up to 60 pounds. These motors run on 12V DC power and can turn at 100 rpm under no load conditions. This motor is represented by Figure 12.

Figure 12: DC Brushed Motor

2.1.6. 12 Volt SLA Battery

The sealed lead acid batteries delivered 12V DC at 12 amp-hour per battery, combined they could power the chassis and components for approximately 4.5 hours per charge (24Ah/5.3A). The rechargeable design allowed for multiple cycles of the batter with very little degradation to the batteries A Seal Lead Acid battery (SLA) is a rechargeable power source which is a particularly important part of any modern vehicle that is used to power any electronic components. The batteries chosen were 12V Sealed Lead Acid batteries, as shown in Figure 13, which were the best option to utilize for this project due to the sheer number of electronic devices that are utilized during the robot's operation. With a capacity of 12Ah (Amp hour), every electronic component in the project is capable of being powered for long hours of operation, debugging, and testing.

Figure 13: 12 Volt Sealed Lead Acid Battery

2.1.7. Power Distribution Board

The power management board provided the distribution from the batteries to the components. It had twin inputs for two batteries to be used in parallel and two switches to allow power to the motors and a main switch for all other components. The voltage regulation was done by a pair of MPM80 Buck Converter Modules and provided the 6.5V power rails and had fuses for all individual power rails. The diagram of the distribution board is shown in Figure 14.

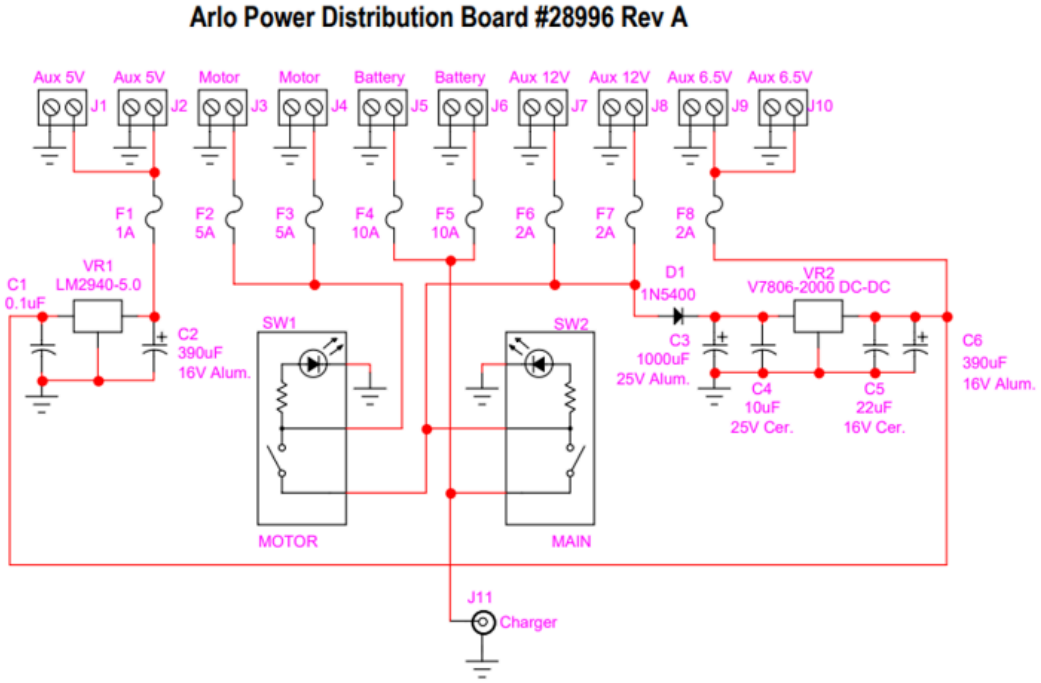

Figure 14: Ario Power Distribution Board

2.2. Software

Below is the list of software used to implement the UGV and the sensor fusion algorithms.

2.2.1. Ubuntu

Ubuntu is an operating system popular in the use of cloud computing. It is free, open source, and has a Linux distribution model based on Debian Architecture and infrastructure.[16] This was chosen as the operating software due to its popularity, reliability, and compatibility with ROS2.

2.2.2. SLAM

SLAM, which stands for Simultaneous Localization and Mapping, is a technique used in robotics and computer vision to enable a robot or an autonomous system to build a map of its

environment while simultaneously localizing itself within that map. The basic idea behind SLAM is to use sensor measurements (such as camera images, laser scans, or range measurements) to estimate both the robot's position and orientation (known as localization) and the positions of landmarks or features in the environment (known as mapping). Below is the SLAM process:

1. Initialization: The SLAM process starts with an initial guess of the robot's position and a basic map. This initial guess can be obtained from sensors like GPS or an inertial measurement unit (IMU).
2. Data Acquisition: The robot moves through the environment, continuously collecting sensor data. This data can come from various sources, such as cameras, lidar, or depth sensors, but for simplicity of the project, 2-D lidar scans were only used. [17]
3. Feature Extraction: The sensor data is processed to extract relevant features or landmarks in the environment. These features could be distinct points in an image, edges, or other distinctive visual or spatial characteristics. [18]
4. Data Association: The extracted features are matched or associated with the features previously stored in the map. This association step determines which features in the current frame correspond to features in the map.
5. State Estimation: Using the associated feature correspondences, the robot's pose (position and orientation) is estimated based on the sensor data. This estimation can be performed using techniques such as filtering, such as Kalman filters, or optimization methods, such as graph-based optimization.
6. Mapping: Once the robot's pose is estimated, the corresponding landmarks' positions are updated or added to the map. The map can be represented in various forms, such as occupancy grids, point clouds, or feature-based representations.
7. Loop Closure: As the robot explores the environment, it may revisit previously seen locations. Loop closure occurs when the SLAM system detects that the robot has returned to a previously visited area. This helps in correcting any accumulated errors and improves the overall map accuracy.
8. Optimization: Periodically, the SLAM system performs optimization to refine the estimated robot poses and landmark positions in the map. Optimization techniques, such as bundle adjustment, are used to minimize the discrepancies between measurements and predictions.

Due to the project's timeline constraints, the implementation of a SLAM algorithm had to be deferred to future development stages. The Autonomous Mobile Robot (AMR) project will assign the task of completing the SLAM algorithm to next year's team. Ample time should be allocated to integrating SLAM, a crucial component for accurate mapping and localization capabilities.

2.2.3. ROS 2

Robot Operating System (ROS) is a flexible and powerful framework designed to facilitate the development of robotic systems [19]. ROS is an open-source project, initially

developed at Stanford University and currently maintained by the Open Robotics organization. Below are some key aspects of the Robot Operating System:

- Modular and Distributed Architecture: ROS follows a modular architecture that promotes code reusability and modularity. It is designed as a collection of independent nodes, which are software components that perform specific tasks. These nodes communicate with each other using a publish-subscribe messaging system, allowing for distribution and concurrent processing.

- Message Passing: Communication between nodes in ROS is based on a publish-subscribe messaging model. Nodes can publish messages on a specific topic, and other nodes that are interested in that topic can subscribe to receive those messages. This decoupled communication mechanism enables loose coupling between modules and facilitates easy integration of different components.

- Package Management: ROS employs a package-based system for organizing and sharing code. A ROS package is a directory that contains libraries, executables, datasets, configuration files, and documentation related to a specific functionality or robot. The package management system allows users to easily download, install, and share packages, promoting code reuse and collaboration.

- Toolset: ROS provides a rich set of tools to aid in the development and debugging of robotic systems. Some of the commonly used tools include:

    - roscore: This is the master process that initializes and manages the ROS communication infrastructure.
    - roslaunch: It is used for launching multiple nodes and setting up the ROS environment.
    - rviz: A 3D visualization tool that allows users to visualize sensor data, robot models, and trajectories.
    - rqt: A framework for developing graphical user interfaces (GUIs) and visualizing data.
    - rosbag: A tool for recording and playing back ROS message data, which is useful for debugging and analysis.

3.2.3 Kalman Filter

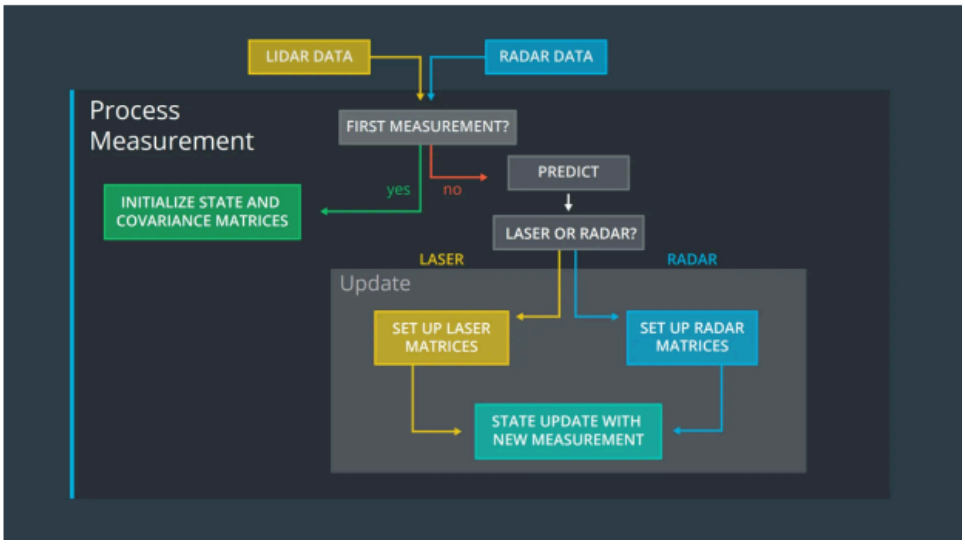

Figure 15: Kalman Filter Process

The Kalman filter is a sensor fusion method that utilizes state observation [20]. As shown in figure 15, state observation means the method can estimate something that cannot be seen or measured directly. This process is divided into 2 phases: Prediction and Update.

When starting the filtering process, it is important to first set up the starting point by initializing state and covariance matrices. Once this is done the first phase of the Kalman filter can begin. The first phase of the Kalman Filter is the Prediction Phase where estimations of the current variables along with their uncertainties can be produced. The second stage is the Update Phase where the next measurements are observed. The estimates are updated based on the weighted average of the predicted state and the state based on the current measurement. A lower weight is given to that with a higher uncertainty.

The Kalman filter does not require the whole history of the past measurements and states to function properly. It only uses the present input measurement and the previously calculated state and uncertainty. Due to the relatively low number of required measurements, the Kalman filter is not computationally intensive and can be executed very quickly. This means it can execute in real time. For these reasons, the Kalman filter is an ideal sensor fusion method for the project and its constraints.

3.4.3 Pycharm

Pycharm is a Python Integrated Development Environment (IDE) [21] that is compatible with Linux systems, such as Ubuntu, which is necessary for implementation onto the Raspberry Pi. It allows for integration of all 5 necessary python modules for Object Detection on one database and can access the USB camera through a video capture command in the code.

2.3. Block Diagram / Flow Chart

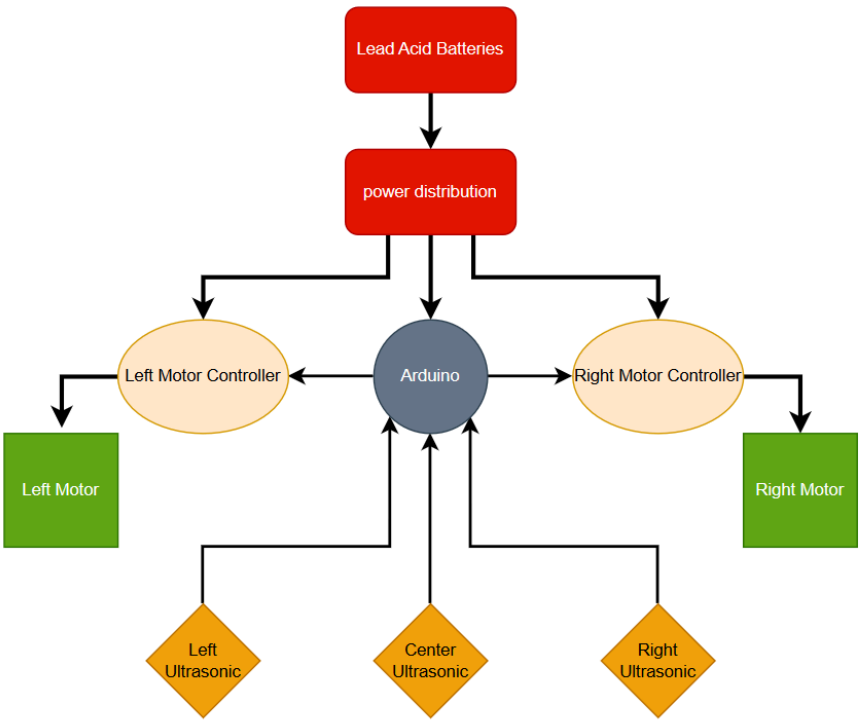

Figure 16: Hardware Block Diagram

Figure 16 is the power distribution and the data flow for the chassis. The batteries supply the 12V DC power to the distribution board where it flows to both motor controllers and Arduino. The Arduino supplies voltage to the ultrasonic sensors and receives data back from the ultrasonic sensors in the form of binary data. The Arduino sends pulse width module (PWM) data to the motor controllers and those commands allow the motor to receive power and turn at a given rate dictated by the (PWM).

Figure 17 represents the data flow chart to the UGV. Due to multiple constraints, the only driving sensors currently are multiple ultrasonic sensors. The lidar and camera do not have any input on how the vehicle drives. The procedure starts with the three onboard ultrasonic sensors detecting measurements. Depending on the distance of the center sensor the vehicle will drive at full speed (above 150cm), half speed (between 50-150 cm), and stop (below 50cm). When the full speed and half speed choices are selected, the system restarts by taking distance measurements. However, once the vehicle is stopped there are two different options: turn left (if the right distance is greater than left) or turn right (if the right distance is less than left, or if the two sensors are equal). While the ultrasonic sensors operate their tasks the camera sensor and LIDAR are performing their own tasks. The camera is detecting and classifying objects in real time, and the LIDAR is creating a 3d dot map of the area the vehicle is driving in. The tasks performed by the camera and LIDAR were created to incorporate sensor fusion into the system. The camera would determine if the system should use ultrasonic data or LIDAR data depending on the material of the object classified, an example is if a human was detected the LIDAR would be the drive measure.

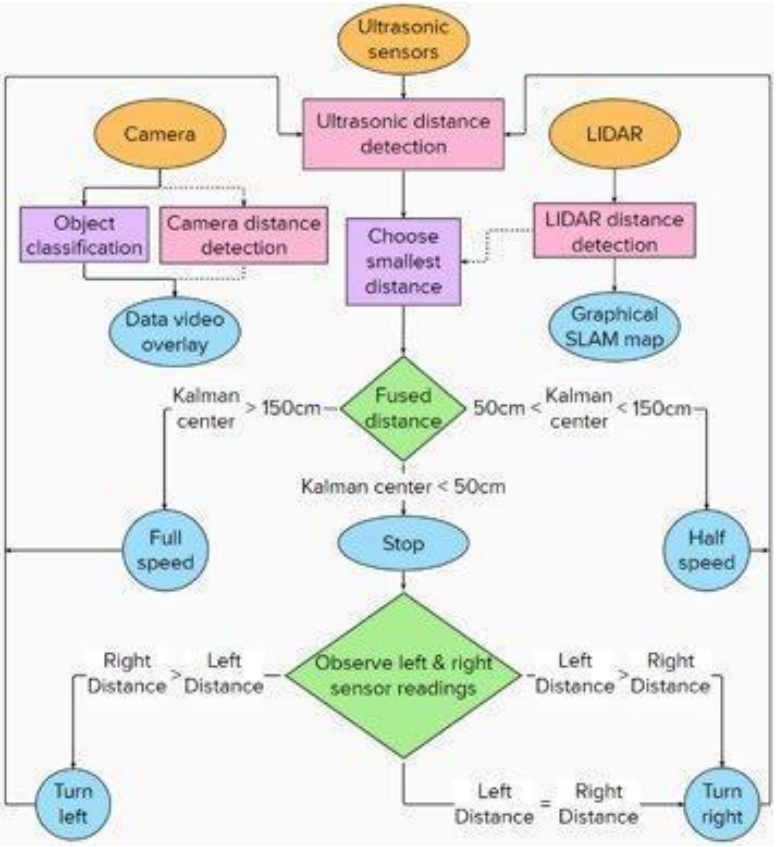

Figure 17: Sensor Fusion Block Diagram

3. Results and Analysis

To develop a quantitative understanding of the advantage of sensor fusion, a series of tests were performed. These tests demonstrate sensor fusion's ability to overcome the specific weaknesses of each sensor and utilize their specific advantages. A trial run was also conducted on the UGV to test its ability to avoid obstacles when in motion.

One reason for having different sensors is to get more accurate sensor data. Each sensor type has a particular strength and weakness. Ultrasonics depend on sound, so they have a weakness against materials with sound-dampening properties such as fabrics. LIDAR depends on laser optics, so it has a weakness to reflective materials such as mirrors. LIDAR and ultrasonics were both tested and compared in their ability to measure object distances up to 2 meters. Both acoustically and optically challenging objects were measured and the fused capability of both sensors is compared to actual distance.

Another reason for having different sensors is to get different types of data. While ultrasonics and LIDAR can capture distance data for an object, a camera sensor can capture images of objects and classify them. The camera was tested in its ability to detect and classify objects at distances up to 2 meters. Classifying objects allows for future implementation of more complex decision-making algorithms.

.

Below are the tests conducted on the UGV:

- Distance measurement testing: Lidar vs Ultrasonic vs Fused Result
- Object detection (and classification) for Camera
- Obstacle avoidance for the UGV

3.1. Distance Measurement

Some sensors perform poorly on objects with certain attributes compared to other sensor types. For example, ultrasonic sensors perform poorly on objects with sound dampening effects such as fabrics since it relies on sound to measure distance. LIDAR does not have that problem since it detects distance based on light. This portion shows the effectiveness of having different sensor types undergo sensor fusion to have a more accurate reading of the environment by testing unusual objects.

3.1.1. Hard Surface

The first test is on a non-reflective, hard object (trashcan) that both Ultrasonics and Lidar should have no problem sensing, as shown in Figure 18. The trashcan is moved to a known distance away from the chassis several times creating 20 data points that show the accuracy of distance measurement. Both the LIDAR and ultrasonic have measurements within ±10 cm accuracy. The data acquired from this test is represented by Figure 19.

Figure 18: Sensors on Chassis measuring the distance of the trashcan.

Figure 19: Lidar vs Ultrasonics vs Actual Distance Measurements of a known Hard Surface Object

3.1.2. Soft Surface

The second test is on a sound-absorbing, soft object (jacket) that is a challenge for ultrasonic, as shown in Figure 20. The LIDAR measurements are within ±10 cm accuracy. The ultrasonics measurements have some accurate measurements but consistently run an error with an approximate 2070 cm reading. Since the specifications of the ultrasonic sensors indicate a range of 2 – 500 cm, all readings outside of that range can be disregarded. All the ultrasonic measurements within the range of operation are within ±10 cm accuracy. The data gathered from this test is represented by Figure 21.

Figure 20: Object measured - a soft fuzzy jacket strapped to a chair.

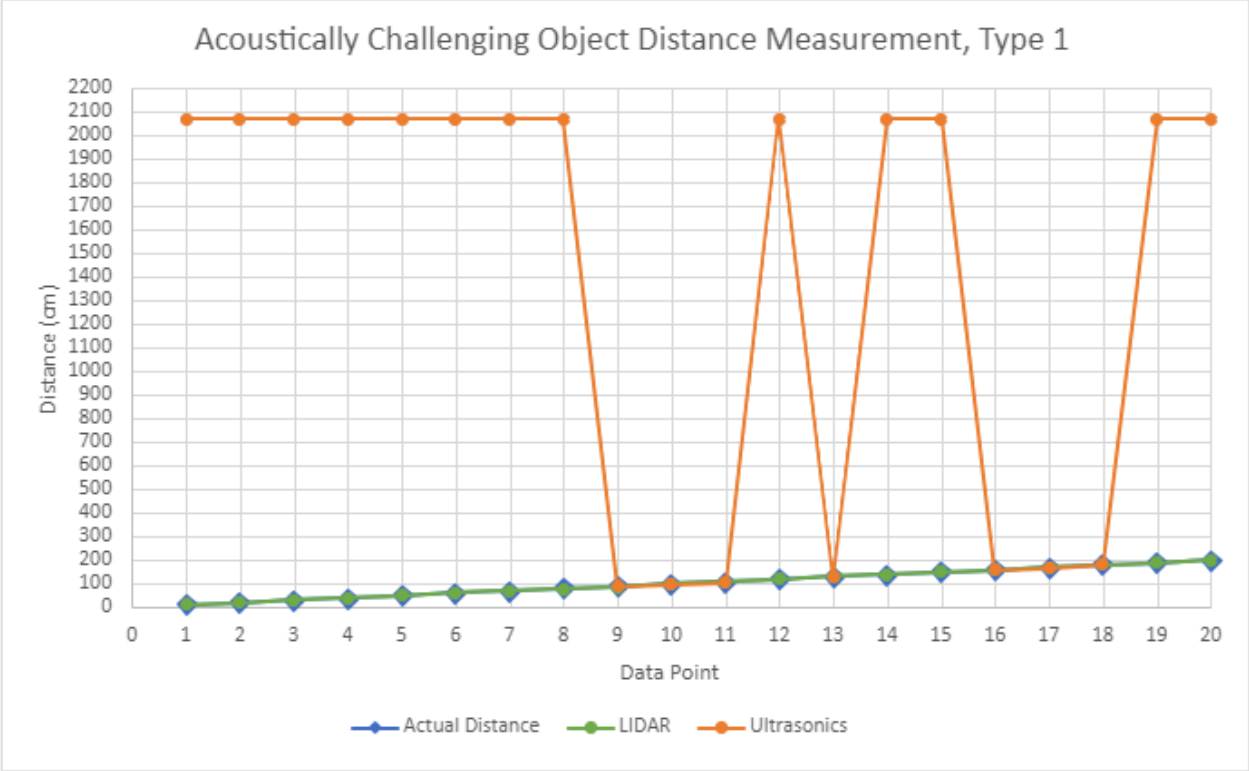

Figure 21: Lidar vs Ultrasonics vs Actual Distance Measurements of a known Sound Dampening Surface Object, Type 1

The third test is of a sound-absorbing, soft object (jacket) with a hard backing (trashcan). This variation of sound-absorbing object is less challenging for ultrasonic, as shown in Figure 22. Again, the LIDAR measurements are within ±10 cm accuracy. Most of the ultrasonic measurements are within ±10 cm accuracy but have an outlier error of 97 cm at the actual distance of 40 cm. This is within the ultrasonic specifications range of 2 – 500 cm. Any

ultrasonic distance errors outside of ±10 cm are all measurements that are greater than the actual distance. The data gathered from this test is represented in Figure 23.

Figure 22: Object measured – Soft fuzzy jacket on a trash bin.

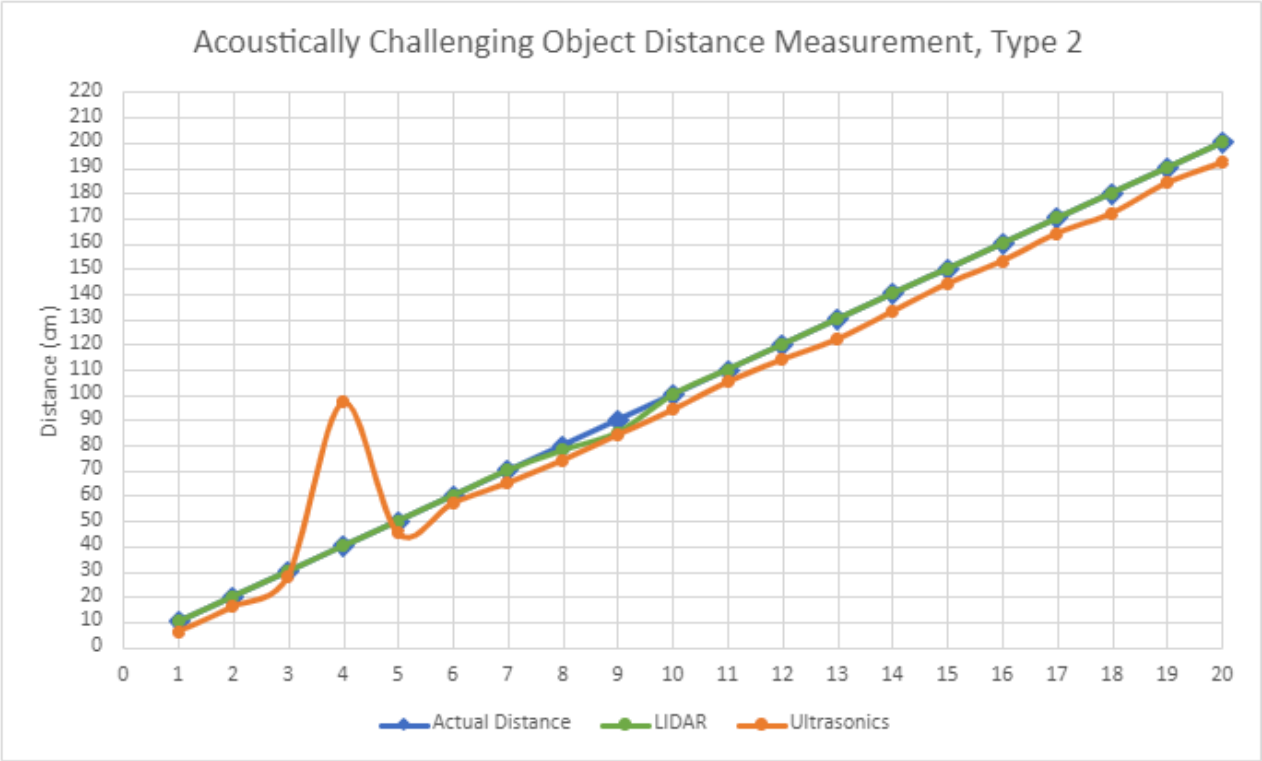

Figure 23: Lidar vs Ultrasonics vs Actual Distance Measurements of a known Sound Dampening Surface Object, Type 2

3.1.3. Reflective Surface

The fourth test is of a reflective object (mirror), as shown in Figure 24. Since LIDAR relies on light-based detection, the light reflects off the mirror and detects a longer path and its readings are significantly greater than the actual distance. Since ultrasonic sensors are sound based, they are immune to optical challenges and have distance measurements within ±10 cm of actual distance. LIDAR completely fails to detect a mirrored object beyond 60 cm. Between 10 – 60 cm, LIDAR measurements have a linear error, consistently overestimating distance by 60 cm. Since the chassis did not change position, its relation to walls and other objects stayed consistent. Thus, the LIDAR measurements have a systematic error because the laser is bouncing off of another static object in the environment that gives a consistent offset. If the LIDAR measurements are adjusted by 60 cm to account for this systematic error, the readings are within ±10 cm of actual distance. The data gathered from this test is represented in Figure 25.

Figure 24: Object measured – Mirror.

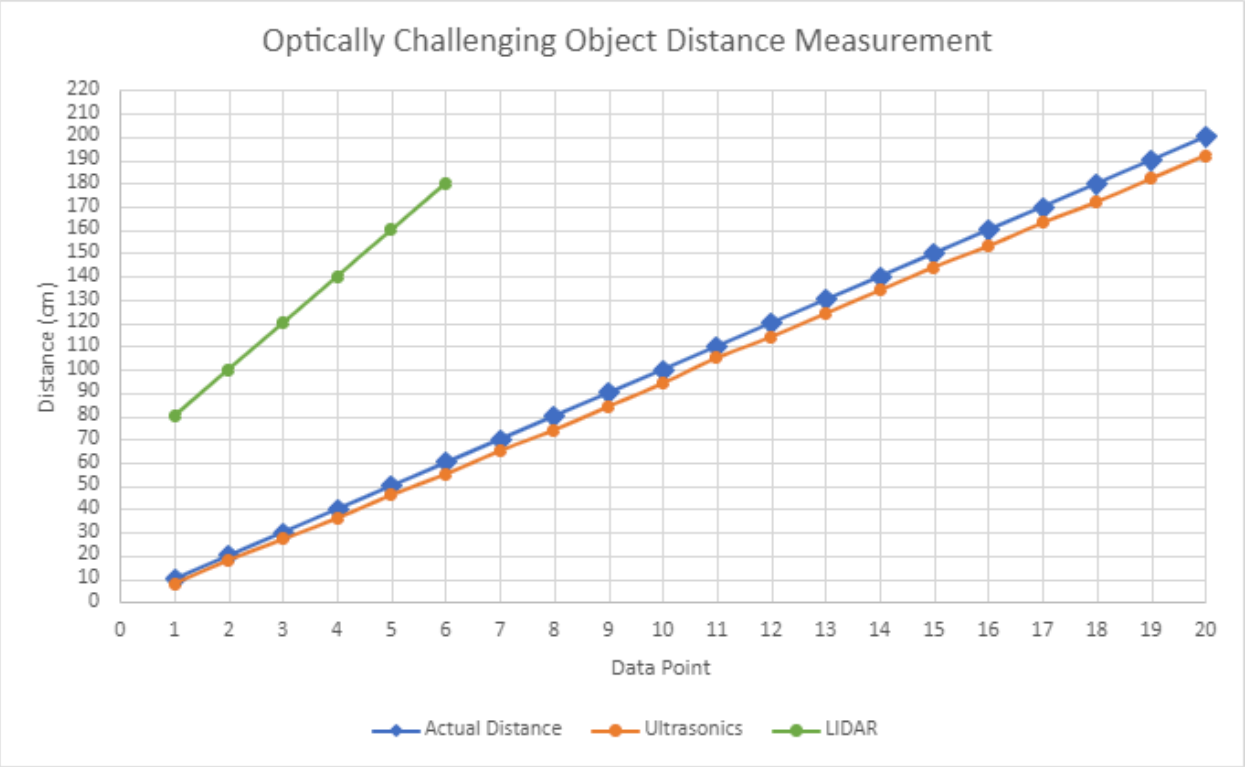

Figure 25: Lidar vs Ultrasonics vs Actual Distance Measurements of a known Reflective Surface Object

3.1.4 Fused Measurement

Sensor fusion for the project was designed according to the results of the individual sensor tests. In all cases where the sensors have a discrepancy larger than ±10 cm, there was systematic error in which distance is always overestimated or out of range. So, fusion in the UGV always takes the smallest stable distance measurement from the sensors to make steering decisions. In regard to statistical error, a Kalman filter is used to average out any small fluctuating errors.

The distance tests were spliced to simulate challenging environments so that the individual sensor measurements can be compared to the fused result. There are a combination of obstacle types and distances to test the ability of fusion to cope with different challenges simultaneously. There were 4 versions of simulated environment:

1. An optically-challenging obstacle between 10 – 60 cm away,
   An acoustically-challenging obstacle, type 1, between 70 – 120 cm away, and
   An acoustically-challenging obstacle, type 2, between 120 – 200 cm away.

2. An acoustically-challenging obstacle, type 2, 10 – 60 cm away,
   An acoustically-challenging obstacle, type 1, between 70 – 120 cm away, and
   An optically-challenging obstacle between 120 – 200 cm away.

3. An optically-challenging obstacle between 10 – 60 cm away,
   An acoustically-challenging obstacle, type 2, between 70 – 120 cm away, and
   An acoustically-challenging obstacle, type 1, between 120 – 200 cm away.

4. An acoustically-challenging obstacle, type 1, between 10 – 60 cm away,
   An optically-challenging obstacle between 70 – 120 cm away, and
   An acoustically-challenging obstacle, type 2, between 120 – 200 cm away.

In all 4 versions of the difficult terrain, the fused result (the red trendline) is always within ±10 cm of actual distance, as shown in Figure 26-29. So, the project demonstrates the success of fusion in increasing the reliability of distance measurements for a UGV.

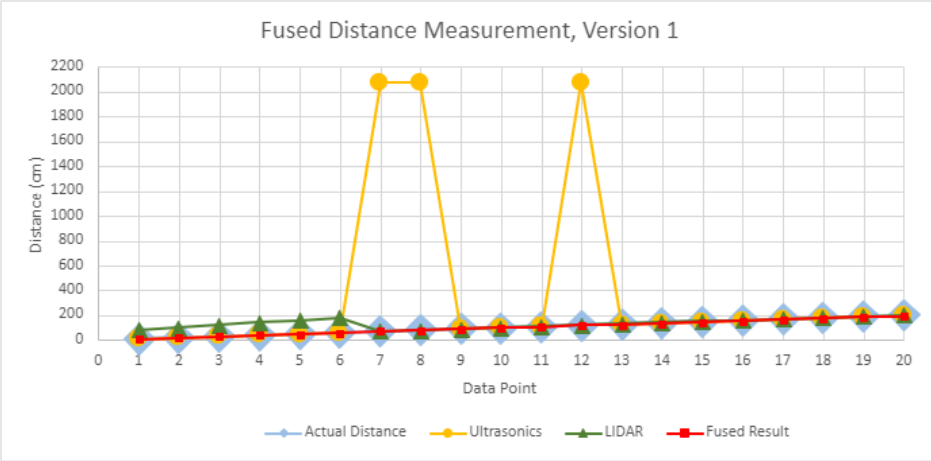

Figure 26: Fused Result Measurements of Composite Testing Environment, Version 1

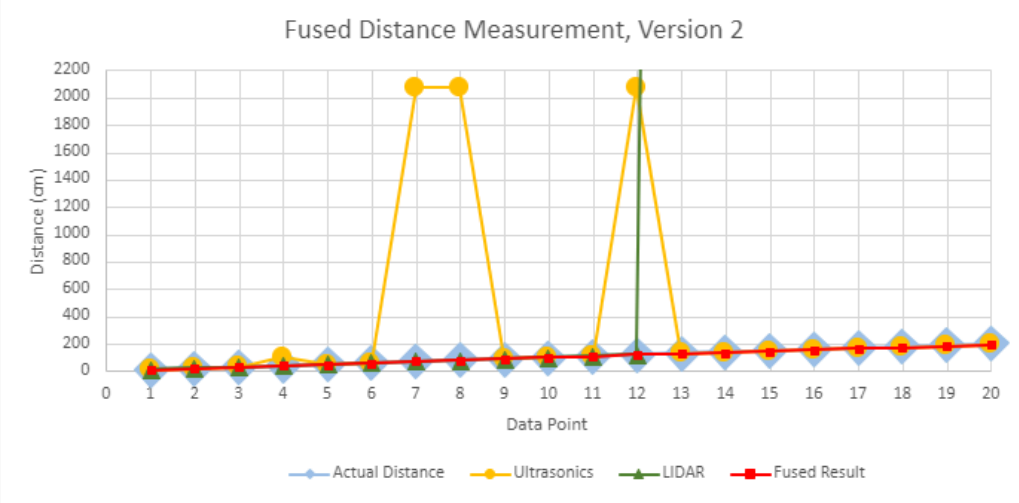

Figure 27: Fused Result Measurements of Composite Testing Environment, Version 2

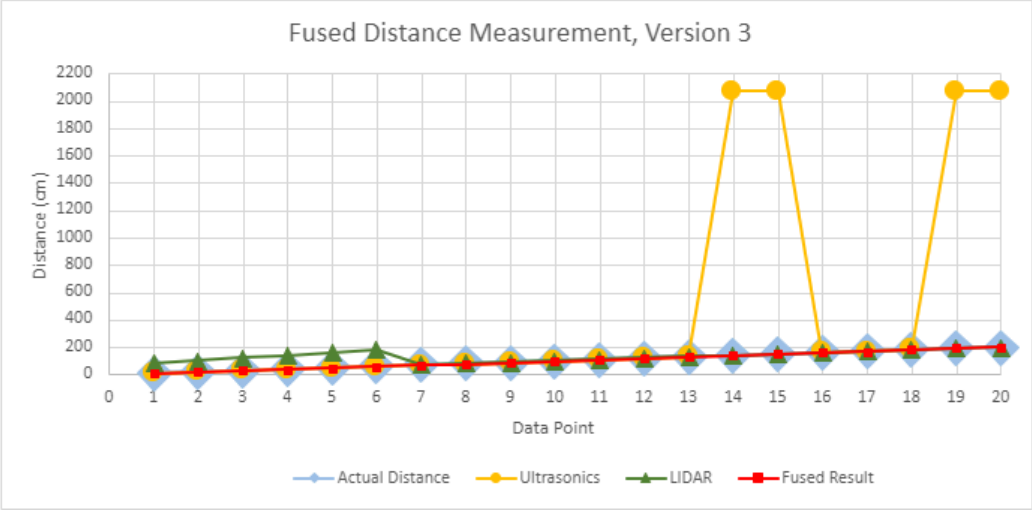

Figure 28: Fused Result Measurements of Composite Testing Environment, Version 3

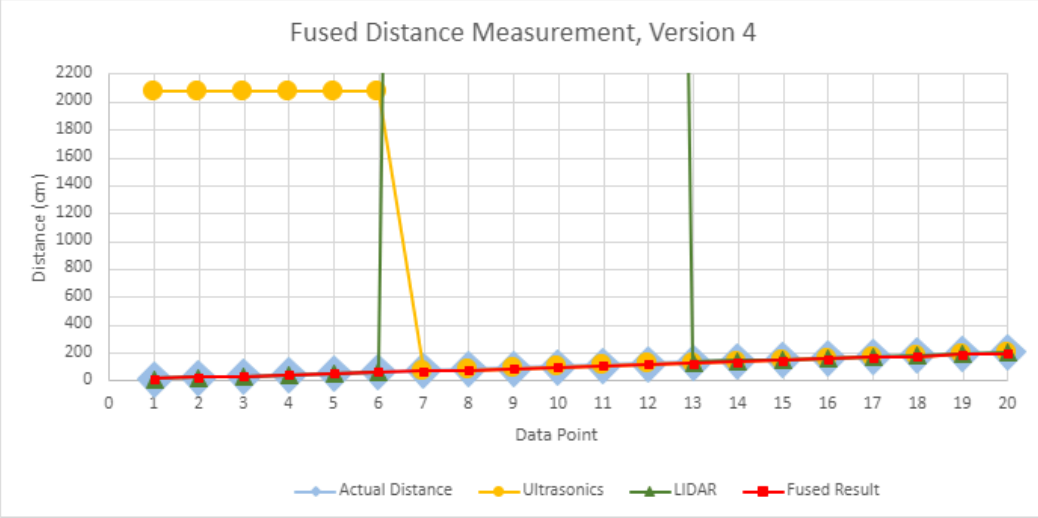

Figure 29: Fused Result Measurements of Composite Testing Environment, Version 4

3.2. Object Detection

Tests were conducted for the camera's ability to detect and identify objects. Figure 30 shows the UGV with the camera attached detecting and identifying objects.

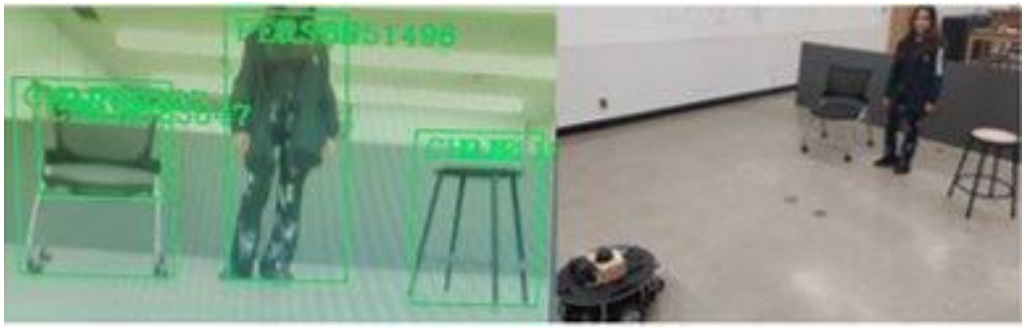

Figure 30: Left: The camera detecting the known objects.
Right: The UGV and several known objects for the camera to detect.

It was decided to use an object detection algorithm that would be most compatible with the algorithm already created for the Ultrasonic Sensors and LiDAR. The code utilized in the project is from *CVZone* "*Object Detection Module using Mobilenet SSD*" [21]. It provides a list of over 90 detectable objects within the file "*coco.names*" which range of objects from persons to airplanes. A small portion of this list is represented in Figure 31.

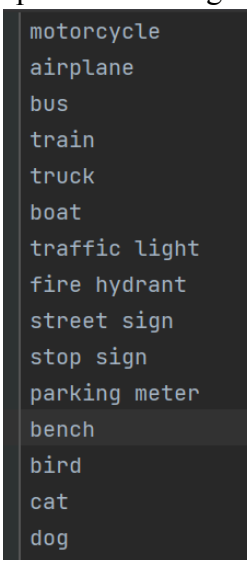

Figure 31: Partial list of detectable objects

The code was narrowed down to a smaller list of specified objects that were used in testing. This was done due to the lack of graphical processing power in the Raspberry Pi 4. The list includes people, chairs, trash bins, bottles, scissors, and smartphones.

A simple if-else statement allows the object classification algorithm to be incorporated into the larger sensor fusion algorithm; The code will output a high, 5V, to a general purpose I/O (GPIO) pin when the camera detects a specified object, and a low signal of 0V when it does not detect anything. The data pin can then be wired to a GPIO pin of the Arduino, which has the code for the ultrasonic sensors and motor drivers, to be set as a flag that rises when an object is seen. The implementation would consist of complementary fusion with the pre-existing Arduino code to verify the existence of an obstacle in the UGV's sight, which would cause the robot to reassess where it will drive to.

To test the maximum distance at which the system could classify objects, tests were conducted to detect and classify scissors and a smartphone from 10, 50, 100, 150, and 200 cm away from the chassis. As seen in figure 32, the machine successfully detects and classifies both the scissors and smartphone at a distance of 50 cm from the chassis.

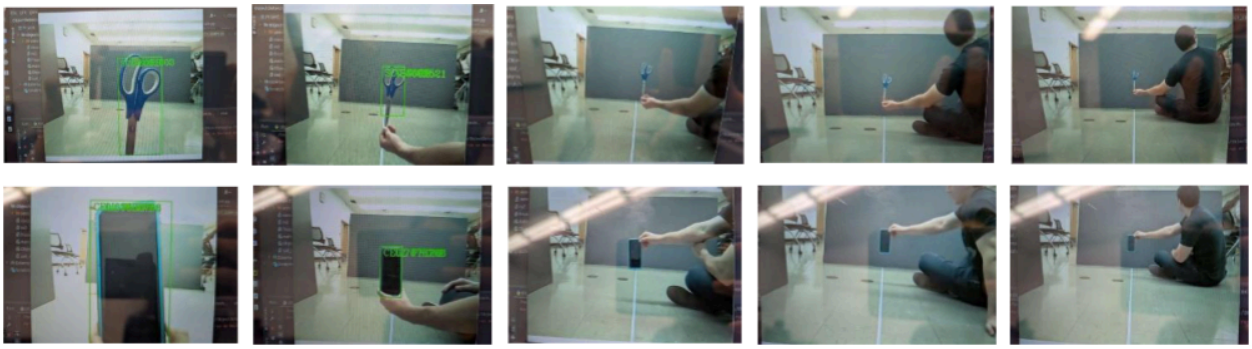

Figure 32: Object Classification Distance Tests, Scissors (Top) and Smartphone (Bottom), from Increasing Distance Left to Right

3.3. Obstacle Avoidance

As shown in Figure 33, the UGV completed a trial run with known obstacles. These obstacles include several trash bins and people with clothing of varying fabrics. The course is represented by Figure 34. The UGV was put into the hallway to run this test.

The UGV went through the entire route without running into any major obstacles. However, it took approximately 15 minutes for the UGV to go the entire route as it was slow moving and made several attempts to U-turn.

Figure 33: Image of the UGV in action

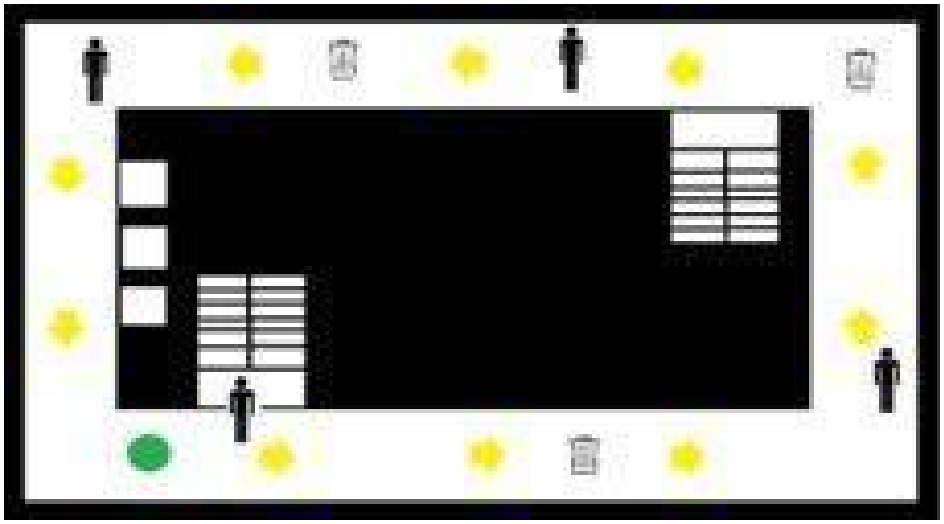

Figure 34: Diagram of the obstacle course (hallway outside of lab) for the UGV the green dot is the UGV, yellow arrows are the direction, and the obstacles are the people and trash bin.

4. Conclusion

Overall, the project was an excellent learning experience that allowed the team to understand how to work in a large group setting. The team gained technical knowledge about the hardware and each of the onboard sensors as well as practical knowledge like communication and dealing with setbacks or issues as they arise.

The team was able to create a UGV with object detection. Although the UGV is slow and the design has limitations, it was able to navigate successfully without crashing into obstacles.

4.1. Final Design

The UGV was a success in demonstrating the utility of sensor fusion for autonomous machines. The project demonstrated three forms of sensor fusion: competitive, complementary, and coordinated fusion. For competitive fusion, LIDAR and ultrasonic overlapped in measurement data so the redundancy increased data dependability. For complementary fusion, the machine's field of detection was increased by using LIDAR and multiple ultrasonics. For coordinated fusion, the camera performed object classification for objects while LIDAR and ultrasonics obtained distance information for objects.

By using competitive fusion, The UGV successfully made distance measurements within ±10 cm of actual distance from the front of the chassis. Complementary fusion allowed the chassis to detect objects at the highest point on the UGV as well as objects close to the ground. Coordinated fusion was demonstrated by the camera successfully detecting and classifying objects up to 50 cm away.

The final design worked well for the constraints of the project. This design works well for low-to-the-ground UGVs but not taller UGVs like passenger vehicles. Since the LIDAR and ultrasonics are limited in their field of detection in the vertical axis, obstacles that are above the ultrasonics but below the lidar, like a tripwire at a height above the ultrasonics, would not be detected. Since the project was limited to a 2D LIDAR and ultrasonics for distance measurements, the design is optimized for a short chassis. For a taller UGV, either 3D LIDAR or a 3D stereo camera would be necessary. Budget and time restraints also meant the UGV was operating on a computer that processes the sensor data slowly. Thus, the UGV had to be designed with a slower traveling speed to account for reaction times. The reduced traveling speed was adequate to allow the UGV to successfully complete an obstacle avoidance test.

4.2. Future Works

Continued work on this project should include integration of a high resolution stereo camera to improve object classification and include stereo-optical distance measurement.

To perform object classification, PyCharm IDE was originally chosen. It would be able to run an object classification program with a pre-built object library. However, difficulties with installation of PyCharm onto the Raspberry Pi stalled progress. Then, the project was donated a ZED camera, as shown in figure 35. The ZED camera has computer vision libraries and software available through the manufacturer. So, Pycharm was abandoned for the ZED SDK (software development kit). However, the ZED demanded a computer with high processing power and the raspberry pi was not able to execute the computation associated with computer vision. Additionally, a severed connection in the ZED cord was discovered, causing it to no longer work. The ZED was then abandoned and a single-lens camera and Pycharm were used. PyCharm eventually ran on the raspberry pi, and could access the camera and run a simple test program. A prebuilt algorithm, the Mobile net SSD detection network [21] on Pycharm, was used to process camera data and output a live feed with objects outlined in a box. It was then discovered that a higher resolution camera would improve image quality so that objects further than 50 cm could be classified.

Figure 35: ZED Camera

Although stereo-optical distance measurements were not implemented into the Arduino algorithm on time, integration is possible by having the code output a high signal to one of its GPIO pins when it detects an object. This high signal would be wired to a data pin on the Arduino, which scans for a high signal and alters its path in the decision tree when an object is detected within 50 cm. However, the plan to integrate stereo camera distance measurement was scrapped due to issues and limitations including, but not limited to, the camera malfunctioning close to the deadline and inadequate processing power of the computer.

Wheel encoders should be used to track rotation of the motors. Wheel encoders are a simple and accurate tool for localization. The wheel encoder data would be utilized in a PID controller to get an accurate output of wheel angular velocity. In addition to this, the motor controller program could be written as a ROS node which would allow for greater steering control.

Implementing a SLAM algorithm would improve navigation in the UGV. If future teams get IMU and wheel encoder data to the RPI, the robot would be able to run SLAM algorithms and the robot would be capable of way point following, path planning, and path correction.

A user designated location would also improve the project as there is currently no designated course for the UGV. During testing, the UGV made several attempts to U-turn when it encountered an obstacle rather than steer around it. The user designated location could come in the form of text or voice commands.

The single most important change that could be made would be to improve the processing power of the single-board computer. Despite Raspberry Pi's specs and popularity with DIY

programmers, the project still experienced considerable lag. The main driver of lag is the graphical processing required for object detection and tracking since the camera has a delay of 5-10 seconds and can only work at the lowest resolution settings. Future iterations of the project should use the Nvidia Jetson Nano or another comparably powerful computer. The Nvidia Jetson Nano, as shown in Figure 36, would help because the specific model is advertised for "applications like image classification, object detection, segmentation". Without improvement to the processor, all other improvements would be moot because of lack of processing power.

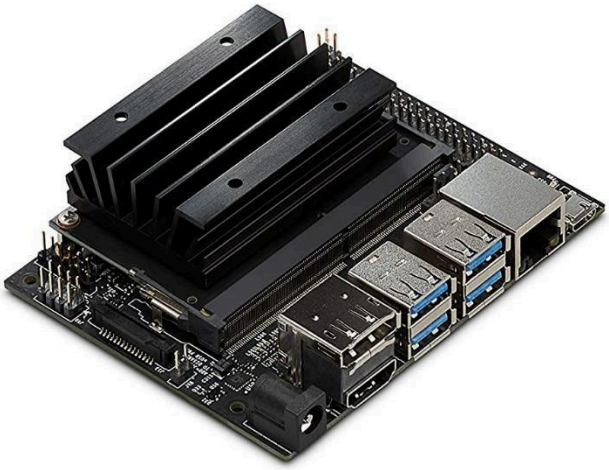

Figure 36: Jetson Namo

5. Reference

[1] “MARG sensors + Bluetooth”, ProjectProto [Online]
http://projectproto.blogspot.com/2011/02/marg-sensors-bluetooth.html
[2] Elmenreich, W. (2002). Sensor Fusion in Time-Triggered Systems, PhD Thesis (PDF). Vienna, Austria: Vienna University of Technology pg 8-9
https://mobile.aau.at/~welmenre/papers/elmenreich_Dissertation_sensorFusionInTimeTriggeredSystems.pdf
[3] Thompson,R. Emery, W.J. “Chapter 4 – The Spatial Analyses of Data Fields”, 2014, Pages 313-424 ScienceDirect (Online)
https://www.sciencedirect.com/science/article/pii/B9780123877826000041
[4] L. Vukonić and M. Tomić, "Ultrasonic Sensors in IoT Applications," 2022 45th Jubilee International Convention on Information, Communication and Electronic Technology (MIPRO), Opatija, Croatia, 2022, pp. 415-420, doi: 10.23919/MIPRO55190.2022.9803772.
[5] Mitchell, R. “Tesla stopped reporting its Autopilot safety numbers online. Why?” Los Angeles Times,
https://www.latimes.com/business/story/2022-12-27/tesla-stopped-reporting-autopilot-safety-statistics-online
[6] Shawn, “Arduino Ultrasonic Sensor Overview and Tutorials”, Seeed Studio [Online]
https://www.seeedstudio.com/blog/2019/11/04/hc-sr04-features-arduino-raspberrypi-guide/
[7] “Ultrasonic Distance Sensor - HC-SR04" Sparkfun [Online]
https://www.sparkfun.com/products/15569
[8] “Lidar” Wikipedia [Online] https://en.wikipedia.org/wiki/Lidar
[9] “Advantages and Disadvantages of LiDAR”, Lidar and Radar Information [Online]
https://lidarradar.com/info/advantages-and-disadvantages-of-lidar
[10] Slamtec, “RPLIDAR Interface Protocol and Application Notes.” Slamtec.com, Shanghai, 06-Apr-2016.
[11] Amazon “Arducam 5MP Camera for Raspberry Pi, 1080P HD OV5647 Camera Module V1 for Pi 4, Raspberry Pi 3, 3B+, and Other A/B Series” Amazon[Online]
https://www.amazon.com/Arducam-Megapixels-Sensor-OV5647-Raspberry/dp/B012V1HEP4
[12] C. Yi, J. Cho “Sensor Fusion for Accurate Ego-Motion Estimation in a Moving Platform”, Research Gate [Online]
https://www.researchgate.net/figure/Roll-pitch-yaw-angles-of-cars-and-other-land-based-vehicles-10_fig2_283951857
[13] “what is an encoder”, Encoders product company
https://www.encoder.com/article-what-is-anencoder#:~:text=A%20beam%20of%20light%20emitted,up%20by%20the%20Photodetector%20Assembly.
[14] Raspberry Pi “Raspberry Pi 4” Raspberry Pi [Online]
https://www.raspberrypi.com/products/raspberry-pi-4-model-b/

[15] Arduino “What is Arduino” Arduino [Online]
https://docs.arduino.cc/learn/starting-guide/whats-arduino
[16] Canonical “The Story of Ubuntu” [Online] https://ubuntu.com/about
[17] SLAM for dummies
https://dspace.mit.edu/bitstream/handle/1721.1/119149/16-412j-spring-2005/contents/projects/1aslam_blas_repo.pdf
[18] Arunabh Ghosh, “Fundamentals of a SLAM algorithm” Grubdragon.github [Online]
https://grubdragon.github.io/erciitb/blog/informative/fundamentals-of-a-slam-algorithm/#:~:text=Steps%20involved%20in%20SLAM%20Algorithms,state%20update%20and%20landmark%20update
[19] S.Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” Science Robotics vol. 7, May 2022.
[20] Kalman Filter
https://medium.com/@mithi/object-tracking-and-fusing-sensor-measurements-using-the-extended-kalman-filter-algorithm-part-1-f2158ef1e4f0 [Online]
[21] M. Hassan, “Object Detection Mobile Net SSD” [Online]
https://www.computervision.zone/courses/object-detection-mobile-net-ssd/

Appendix I: Code - Arduino Ultrasonic detection and drive motor logic states

The following code is the overall drive code for the UGV system. This code is in the Arduino IDE and gathers data from the three onboard ultrasonic sensors through a Kalman filter and controls both motor drivers. The ultrasonic portion on the flowchart represented in Figure 17 is the process of the code. The code drives the motors Full, half, and stop at different ultrasonic readings. Full is over 150 cm, half is between 50 to 150 cm, and stop is below 50 cm. When the motors stop the system then decides whether it should turn left or right. When the right reading is greater than the left reading the system will turn left. When the left is greater, or the two sensors are equal, the system will turn right.

```
#include <Servo.h>
String inputString;
Servo servoLeft;
Servo servoRight;
#define EchoPINC 7                                      // attach pin D2 Arduino to pin Echo of HC-SR04
#define TriggerPINC 4                                    //attach pin D3 Arduino to pin Trig of HC-SR04
#define EchoPINL 9                                      // attach pin D2 Arduino to pin Echo of HC-SR04
#define TriggerPINL 6                                    //attach pin D3 Arduino to pin Trig of HC-SR04
#define EchoPINR 8                                      // attach pin D2 Arduino to pin Echo of HC-SR04
#define TriggerPINR 5                                    //attach pin D3 Arduino to pin Trig of HC-SR04
int speedL=1500;                                  //initial speed setting
int speedR=1500;                                  //initial speed setting

//**********************GLOBALS ABOVE*************************
int kalmanC(int U){
  static const double R = 40;
  static const double H = 1.00;
  static double Q = 10;
  static double P = 0;
  static double U_hat = 0;
  static double K = 0;
  K = P*H/(H*P*H+R);
  U_hat += + K*(U-H*U_hat);
  P = (1-K*H)*P+Q;
  return U_hat;
}
```

```
int kalmanL(int U){
  static const double R = 40;
  static const double H = 1.00;
  static double Q = 10;
  static double P = 0;
  static double U_hat = 0;
  static double K = 0;
  K = P*H/(H*P*H+R);
  U_hat += + K*(U-H*U_hat);
  P = (1-K*H)*P+Q;
  return U_hat;
}

int kalmanR(int U){
  static const double R = 40;
  static const double H = 1.00;
  static double Q = 10;
  static double P = 0;
  static double U_hat = 0;
  static double K = 0;
  K = P*H/(H*P*H+R);
  U_hat += + K*(U-H*U_hat);
  P = (1-K*H)*P+Q;
  return U_hat;
}

//******************KALMAN FILTER CALL ABOVE*************************
int distC(){                                          //CALL TO GET CENTER DISTANCE MEASUREMENT
  digitalWrite(TriggerPINC,LOW);                            //Sets TriggerPIN1 LOW for 2 microseconds
  delayMicroseconds(2);
  digitalWrite(TriggerPINC,HIGH);                           //Sets TriggerPIN1 HIGH for 2 microseconds
  delayMicroseconds(2);
  digitalWrite(TriggerPINC,LOW);                            //Sets TriggerPIN1 LOW

  long timedelayC = pulseIn(EchoPINC,HIGH);                 //Reads EchoPIN1 and returns sound wave travel time in microseconds
  int distanceC = 0.0343 * (timedelayC/2);                  //Calculates the reading to give the distance in cm under newdistance1
  distanceC = kalmanC(distanceC);                            //Center distance measurement
  return distanceC;
}

int distL(){                                          //CALL TO GET LEFT DISTANCE MEASUREMENT
  digitalWrite(TriggerPINL,LOW);                            //Sets TriggerPIN1 LOW for 2 microseconds for Left
  delayMicroseconds(2);
  digitalWrite(TriggerPINL,HIGH);                           //Sets TriggerPIN1 HIGH for 2 microseconds for Left
  delayMicroseconds(2);
  digitalWrite(TriggerPINL,LOW);                            //Sets TriggerPIN1 LOW for Left

  long timedelayL = pulseIn(EchoPINL,HIGH);                 //Reads EchoPIN2 and returns sound wave travel time in microseconds
  int distanceL = 0.0343 * (timedelayL/2);                  //Calculates the reading to give the distance in cm under newdistance1
  distanceL = kalmanL(distanceL);
  return distanceL;
}

int distR(){                                          //CALL TO GET RIGHT DISTANCE MEASUREMENT
  digitalWrite(TriggerPINR,LOW);                            //Sets TriggerPIN1 LOW for 2 microseconds for Right
  delayMicroseconds(2);
  digitalWrite(TriggerPINR,HIGH);                           //Sets TriggerPIN1 HIGH for 2 microseconds fir Right
  delayMicroseconds(2);
  digitalWrite(TriggerPINR,LOW);                            //Sets TriggerPIN1 LOW for Right

  long timedelayR = pulseIn(EchoPINR,HIGH);                 //Reads EchoPIN3 and returns sound wave travel time in microseconds
  int distanceR = 0.0343 * (timedelayR/2);                  //Calculates the reading to give the distance in cm under newdistance1
  distanceR = kalmanR(distanceR);
  return distanceR;
}

//************************MAIN VOID BELOW*************************************
void setup() {
 Serial.begin(9600);                                        //Enable serial port data, sets rate at 9600 bps
 servoLeft.attach(11);                                       //Initalize port 9 to be Left motorcontroller
 servoRight.attach(3);                                      //Initalize port 10 to be Right motorcontroller
 servoLeft.writeMicroseconds(1500);                         //Initalize Left motor contoller stop
 servoRight.writeMicroseconds(1500);                        //Initalize Right motor contoller stop
 pinMode(TriggerPINC, OUTPUT);                              // Sets the trigPin as an OUTPUT
 pinMode(EchoPINC, INPUT);                                  // Sets the echoPin as an INPUT
 pinMode(TriggerPINL, OUTPUT);                              // Sets the trigPin as an OUTPUT
 pinMode(EchoPINL, INPUT);                                  // Sets the echoPin as an INPUT
 pinMode(TriggerPINR, OUTPUT);                              // Sets the trigPin as an OUTPUT
 pinMode(EchoPINR, INPUT);                                  // Sets the echoPin as an INPUT
}
```

```
void loop() {
  int kalmandistC=distC();                          //Center distance measurement
  int kalmandistL=distL();                          //Left distance measurement
  int kalmandistR=distR();                          //Right distance measurement

  Serial.print("Sensor Center: ");
  Serial.print(kalmandistC);
  Serial.print(" ||Sensor Right: ");
  Serial.print(kalmandistR);
  Serial.print(" ||Sensor Left: ");
  Serial.print(kalmandistL);
  Serial.println(" ||");

//********************MOTOR SPEED CONTOL BELOW*****************
  if (kalmandistC<=50 || kalmandistR<=100 || kalmandistL<=100){     //Full stop both motors***************
    servoLeft.writeMicroseconds(speedL=1500);
    servoRight.writeMicroseconds(speedR=1500);
    speedL=1500;
    speedR=1500;


      if (kalmandistR > kalmandistL){                          //Turn Left statement
        while(kalmandistL<300){
          servoLeft.writeMicroseconds(1600);
          servoRight.writeMicroseconds(1400);
          delay(20);
          int kalmandistL=distL();                            //Left distance measurement update
          Serial.print(" ||Sensor Left turn: ");
          Serial.print(kalmandistL);
          Serial.println(" ||");
          if(kalmandistL>500){
            break;
          }
        }
      }

      else if (kalmandistL > kalmandistR){                     //Turn Right statement
        while(kalmandistR<300){
          servoLeft.writeMicroseconds(1400);
          servoRight.writeMicroseconds(1600);
          delay(20);
          int kalmandistR=distR();                            //Right distance measurement update
          Serial.print(" ||Sensor Right turn: ");
          Serial.print(kalmandistR);
          Serial.println(" ||");
        if(kalmandistR>500){
          break;
        }
        }
      }
    }
  }

  else if (kalmandistC > 50 && kalmandistC <=150){             //Half forward both motors**********
    if(speedL==1500){
      for(int speed=0; speed<=100; speed+=10){                    //speed up wheels to half speed
        servoLeft.writeMicroseconds(speedL+speed);
        servoRight.writeMicroseconds(speedR+speed);
        delay(40);
      }
      speedL=1600;
      speedR=1600;
    }
    else if(speedL==1700){
      for(int speed=0; speed<=100; speed+=10){                    //slow down wheels to half speed
        servoLeft.writeMicroseconds(speedL-speed);
        servoRight.writeMicroseconds(speedR-speed);
        delay(40);
      }
      speedL=1600;
      speedR=1600;
    }
      else{
        delay(20);
      }
```

```
}

else if (kalmandistC > 150){                          //Full forward both motors***********
  for(int speed=0; speed<=200; speed+=2){
    servoLeft.writeMicroseconds(speedL+speed);
    servoRight.writeMicroseconds(speedR+speed);
    delay(40);
  }
  speedL=1700;
  speedR=1700;
}
```

Appendix II: Code - AMR URDF

The Unified Robotic Description Format (URDF) is an XML file format used in ROS to describe all elements of a robot. The URDF file is used to represent the robot's links, joints, sensors, and other relevant properties. It allows users to define the geometry, kinematics, dynamics, and visual appearance of the robot. Throughout this project this file is used in conjunction with other ros packages and softwares in order to test and run simulations, make a real time GUI of the lidar data or reference it to perform SLAM or navigation calculations. The first 152 lines of code describe the physical aspects of the robot. This includes the geometry, colors, mass, interia, center of gravity origin and coordinates in space. Each part and how they connect with other parts can be described as fixed, prismatic, revolute or continuous and each link in relation to another is called a parent or child. This parent and child relationship is used to describe the cascading motion of every single part. So when the AMR parent base rotates 90 degrees all the child links will be rotated 90 degrees too. Every part of the AMR is defined as fixed except for the axel which is revolute. Line 154 to line 385 describes the placement of all the sensors in space and where they attach to the AMR as well as some configuration variables for the gazebo simulation. With the complete URDF next years team will be able to fully utilize the NAV2 package in order to implement SLAM, course correction, way point tracking or any other algorithms in NAV2.

```
<?xml version="1.0"?>
<robot name="sam_bot" xmlns:xacro="http://ros.org/wiki/xacro">

  <!-- Define robot constants -->
  <xacro:property name="base_width" value="0.31"/>
  <xacro:property name="base_length" value="0.42"/>
  <xacro:property name="base_height" value="0.18"/>

  <xacro:property name="wheel_radius" value="0.10"/>
  <xacro:property name="wheel_width" value="0.04"/>
  <xacro:property name="wheel_ygap" value="0.025"/>
  <xacro:property name="wheel_zoff" value="0.05"/>
  <xacro:property name="wheel_xoff" value="0.12"/>

  <xacro:property name="caster_xoff" value="0.14"/>

  <!-- Define some commonly used intertial properties  -->
  <xacro:macro name="box_inertia" params="m w h d">
    <inertial>
      <origin xyz="0 0 0" rpy="${pi/2} 0 ${pi/2}"/>
      <mass value="${m}"/>
      <inertia ixx="${(m/12) * (h*h + d*d)}" ixy="0.0" ixz="0.0" iyy="${(m/12) * (w*w + d*d)}" iyz="0.0" izz="${(m/12) * (w*w + h*h)}"/>
    </inertial>
  </xacro:macro>

  <xacro:macro name="cylinder_inertia" params="m r h">
    <inertial>
      <origin xyz="0 0 0" rpy="${pi/2} 0 0" />
      <mass value="${m}"/>
      <inertia ixx="${(m/12) * (3*r*r + h*h)}" ixy = "0" ixz = "0" iyy="${(m/12) * (3*r*r + h*h)}" iyz = "0" izz="${(m/2) * (r*r)}"/>
    </inertial>
  </xacro:macro>

  <xacro:macro name="sphere_inertia" params="m r">
    <inertial>
      <mass value="${m}"/>
      <inertia ixx="${(2/5) * m * (r*r)}" ixy="0.0" ixz="0.0" iyy="${(2/5) * m * (r*r)}" iyz="0.0" izz="${(2/5) * m * (r*r)}"/>
    </inertial>
  </xacro:macro>

  <!-- Robot Base -->
  <link name="base_link">
    <visual>
      <geometry>
        <box size="${base_length} ${base_width} ${base_height}"/>
      </geometry>
      <material name="Cyan">
        <color rgba="0 1.0 1.0 1.0"/>
      </material>
    </visual>

    <collision>
      <geometry>
        <box size="${base_length} ${base_width} ${base_height}"/>
      </geometry>
    </collision>

    <xacro:box_inertia m="15" w="${base_width}" d="${base_length}" h="${base_height}"/>
  </link>
```

```
60
61     <!-- Robot Footprint -->
62    <link name="base_footprint">
63          <xacro:box_inertia m="0" w="0" d="0" h="0"/>
64    </link>
65
66    <joint name="base_joint" type="fixed">
67      <parent link="base_link"/>
68      <child link="base_footprint"/>
69      <origin xyz="0.0 0.0 ${-(wheel_radius+wheel_zoff)}" rpy="0 0 0"/>
70    </joint>
71
72    <!-- Wheels -->
73    <xacro:macro name="wheel" params="prefix x_reflect y_reflect">
74      <link name="${prefix}_link">
75        <visual>
76          <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
77          <geometry>
78              <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
79          </geometry>
80          <material name="Gray">
81            <color rgba="0.5 0.5 0.5 1.0"/>
82          </material>
83        </visual>
84
85        <collision>
86          <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
87            <geometry>
88              <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
89            </geometry>
90          </collision>
91
92          <xacro:cylinder_inertia m="0.5" r="${wheel_radius}" h="${wheel_width}"/>
93        </link>
94
95        <joint name="${prefix}_joint" type="continuous">
96          <parent link="base_link"/>
97          <child link="${prefix}_link"/>
98          <origin xyz="${x_reflect*wheel_xoff} ${y_reflect*(base_width/2+wheel_ygap)} ${-wheel_zoff}" rpy="0 0 0"/>
99          <axis xyz="0 1 0"/>
100       </joint>
101     </xacro:macro>
102
103     <xacro:wheel prefix="drivewhl_l" x_reflect="-1" y_reflect="1" />
104     <xacro:wheel prefix="drivewhl_r" x_reflect="-1" y_reflect="-1" />
105
106     <link name="front_caster">
107       <visual>
108         <geometry>
109           <sphere radius="${(wheel_radius+wheel_zoff-(base_height/2))}"/>
110         </geometry>
111         <material name="Cyan">
112           <color rgba="0 1.0 1.0 1.0"/>
113         </material>
114       </visual>
```

```
115
116       <collision>
117         <origin xyz="0 0 0" rpy="0 0 0"/>
118         <geometry>
119           <sphere radius="${(wheel_radius+wheel_zoff-(base_height/2))}"/>
120         </geometry>
121       </collision>
122
123       <xacro:sphere_inertia m="0.5" r="${(wheel_radius+wheel_zoff-(base_height/2))}"/>
124     </link>
125
126     <joint name="caster_joint" type="fixed">
127       <parent link="base_link"/>
128       <child link="front_caster"/>
129       <origin xyz="${caster_xoff} 0.0 ${-(base_height/2)}" rpy="0 0 0"/>
130     </joint>
131
132     <link name="imu_link">
133       <visual>
134         <geometry>
135           <box size="0.1 0.1 0.1"/>
136         </geometry>
137       </visual>
138
139       <collision>
140         <geometry>
141           <box size="0.1 0.1 0.1"/>
142         </geometry>
143       </collision>
144
145       <xacro:box_inertia m="0.1" w="0.1" d="0.1" h="0.1"/>
146     </link>
147
148     <joint name="imu_joint" type="fixed">
149       <parent link="base_link"/>
150       <child link="imu_link"/>
151       <origin xyz="0 0 0.01"/>
152     </joint>
153
154     <gazebo reference="imu_link">
155       <sensor name="imu_sensor" type="imu">
156         <plugin filename="libgazebo_ros_imu_sensor.so" name="imu_plugin">
157           <ros>
158             <namespace>/demo</namespace>
159             <remapping>~/out:=imu</remapping>
160           </ros>
161           <initial_orientation_as_reference>false</initial_orientation_as_reference>
162         </plugin>
163         <always_on>true</always_on>
164         <update_rate>100</update_rate>
165         <visualize>true</visualize>
166        <imu>
167           <angular_velocity>
168             <x>
169               <noise type="gaussian">
170                 <mean>0.0</mean>
```

```
            <stddev>2e-4</stddev>
            <bias_mean>0.0000075</bias_mean>
            <bias_stddev>0.0000008</bias_stddev>
          </noise>
        </x>
        <y>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>2e-4</stddev>
            <bias_mean>0.0000075</bias_mean>
            <bias_stddev>0.0000008</bias_stddev>
          </noise>
        </y>
        <z>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>2e-4</stddev>
            <bias_mean>0.0000075</bias_mean>
            <bias_stddev>0.0000008</bias_stddev>
          </noise>
        </z>
      </angular_velocity>
      <linear_acceleration>
        <x>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>1.7e-2</stddev>
```

```
              <bias_mean>0.1</bias_mean>
              <bias_stddev>0.001</bias_stddev>
            </noise>
          </x>
          <y>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>1.7e-2</stddev>
              <bias_mean>0.1</bias_mean>
              <bias_stddev>0.001</bias_stddev>
            </noise>
          </y>
          <z>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>1.7e-2</stddev>
              <bias_mean>0.1</bias_mean>
              <bias_stddev>0.001</bias_stddev>
            </noise>
          </z>
        </linear_acceleration>
      </imu>
    </sensor>
  </gazebo>

  <gazebo>
    <plugin name='diff_drive' filename='libgazebo_ros_diff_drive.so'>
      <ros>
        <namespace>/demo</namespace>
      </ros>

      <!-- wheels -->
      <left_joint>drivewhl_l_joint</left_joint>
      <right_joint>drivewhl_r_joint</right_joint>

      <!-- kinematics -->
      <wheel_separation>0.4</wheel_separation>
      <wheel_diameter>0.2</wheel_diameter>

      <!-- limits -->
      <max_wheel_torque>20</max_wheel_torque>
      <max_wheel_acceleration>1.0</max_wheel_acceleration>

      <!-- output -->
      <publish_odom>true</publish_odom>
      <publish_odom_tf>false</publish_odom_tf>
      <publish_wheel_tf>true</publish_wheel_tf>

      <odometry_frame>odom</odometry_frame>
      <robot_base_frame>base_link</robot_base_frame>
    </plugin>
  </gazebo>

<link name="lidar_link">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0"/>
```

```
      <mass value="0.125"/>
      <inertia ixx="0.001"  ixy="0"  ixz="0" iyy="0.001" iyz="0" izz="0.001" />
    </inertial>

    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
         <cylinder radius="0.0508" length="0.055"/>
      </geometry>
    </collision>

    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
         <cylinder radius="0.0508" length="0.055"/>
      </geometry>
    </visual>
  </link>

  <joint name="lidar_joint" type="fixed">
    <parent link="base_link"/>
    <child link="lidar_link"/>
    <origin xyz="0 0 0.12" rpy="0 0 0"/>
  </joint>

  <gazebo reference="lidar_link">
    <sensor name="lidar" type="ray">
      <always_on>true</always_on>
      <visualize>true</visualize>
      <update_rate>5</update_rate>
      <ray>
        <scan>
          <horizontal>
            <samples>360</samples>
            <resolution>1.000000</resolution>
            <min_angle>0.000000</min_angle>
            <max_angle>6.280000</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.120000</min>
          <max>3.5</max>
          <resolution>0.015000</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </ray>
      <plugin name="scan" filename="libgazebo_ros_ray_sensor.so">
        <ros>
          <remapping>~/out:=scan</remapping>
        </ros>
        <output_type>sensor_msgs/LaserScan</output_type>
```

```
        <frame_name>lidar_link</frame_name>
      </plugin>
    </sensor>
  </gazebo>

  <link name="camera_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.015 0.130 0.022"/>
      </geometry>
    </visual>

    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.015 0.130 0.022"/>
      </geometry>
    </collision>

    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <mass value="0.035"/>
      <inertia ixx="0.001"  ixy="0"  ixz="0" iyy="0.001" iyz="0" izz="0.001" />
    </inertial>
  </link>

  <joint name="camera_joint" type="fixed">
    <child link="camera_link"/>
    <origin xyz="0.215 0 0.05" rpy="0 0 0"/>
  </joint>

  <link name="camera_depth_frame"/>

  <joint name="camera_depth_joint" type="fixed">
    <origin xyz="0 0 0" rpy="${-pi/2} 0 ${-pi/2}"/>
    <parent link="camera_link"/>
    <child link="camera_depth_frame"/>
  </joint>

  <gazebo reference="camera_link">
    <sensor name="depth_camera" type="depth">
      <visualize>true</visualize>
      <update_rate>30.0</update_rate>
      <camera name="camera">
        <horizontal_fov>1.047198</horizontal_fov>
        <image>
          <width>640</width>
          <height>480</height>
          <format>R8G8B8</format>
        </image>
        <clip>
          <near>0.05</near>
          <far>3</far>
        </clip>
```

```
365         </camera>
366         <plugin name="depth_camera_controller" filename="libgazebo_ros_camera.so">
367           <baseline>0.2</baseline>
368           <alwaysOn>true</alwaysOn>
369           <updateRate>0.0</updateRate>
370           <frame_name>camera_depth_frame</frame_name>
371           <pointCloudCutoff>0.5</pointCloudCutoff>
372           <pointCloudCutoffMax>3.0</pointCloudCutoffMax>
373           <distortionK1>0</distortionK1>
374           <distortionK2>0</distortionK2>
375           <distortionK3>0</distortionK3>
376           <distortionT1>0</distortionT1>
377           <distortionT2>0</distortionT2>
378           <CxPrime>0</CxPrime>
379           <Cx>0</Cx>
380           <Cy>0</Cy>
381           <focalLength>0</focalLength>
382           <hackBaseline>0</hackBaseline>
383         </plugin>
384       </sensor>
385     </gazebo>
386   </robot>
```